

Today's Agenda

- Copy Constructor
 - Shallow Copy and Deep Copy
 - Static Members
 - Operator Overloading and Friend Function
 - Association
 - Inheritance
 - Basic Inheritance
 - Protected Data Member
 - Diamond Problem and Mode of Inheritance
 - Shadowing
 - Object Slicing
 - Upcasting and Downcasting
 - Virtual Function
 - Pure Virtual Function and Abstract Class
-

1. Copy Constructor

A **copy constructor** is a special constructor that initializes a new object as a copy of an existing object of the same class.

Syntax:

```
ClassName(const ClassName& obj);
```

When Is the Copy Constructor Called?

The copy constructor is invoked in **5 situations**:

1. When you initialize an object using an existing object:

```
class Test {  
    int num1;  
public:  
    Test(int n) { num1 = n; }  
};  
  
Test t1(10);  
Test t2 = t1; // Copy constructor is called
```

2. When an object is passed by value to a function:

```
void display(Test t1); // Pass by value

display(t1); // Copy constructor is called here
```

3. When an object is returned by value from a function:

```
Test createObject() {
    Test t(5);
    return t; // Copy constructor is called here
}
```

4. When an object is thrown as an exception:

```
throw t1; // Copy constructor is called
```

5. When an object is caught by value in a catch block:

```
catch (Test t) {
    // Copy constructor is called when exception is caught
}
```

Key Point: By default, the compiler provides a copy constructor that performs a **shallow copy**.

2. Shallow Copy vs Deep Copy

Shallow Copy

A **shallow copy** copies the content of one object into another **as-is** — field by field. This means both objects end up pointing to the **same memory location** if any pointer members are involved.

```
class Complex {
    int* ptrReal;
    int imag;
public:
    Complex(const Complex& obj) {
        this->ptrReal=obj.real; // New allocation

        this->imag=obj.imag; // Copy content
    }
    // Default copy constructor = shallow copy (DANGEROUS with pointers)
};
```

```
Complex c1;
Complex c2 = c1; // Both c1.real and c2.real point to the SAME memory!
```

Problem: When either object is destroyed, the destructor frees the shared memory. The other object is now left with a **dangling pointer** — this is called a **double free** error.

Deep Copy

A **deep copy** creates a **new independent copy** of all dynamically allocated memory. Each object owns its own memory.

```
class Complex {
    int* ptrReal;
    int imag;
public:
    Complex(int real=0,int imag=0) {
        this->ptrReal=new int;
        *ptrReal=real;
        this->imag=imag;
    }

    // Deep copy constructor
    Complex(const Complex& obj) {
        this->ptrReal=new int; // New allocation
        *ptrReal=*obj;
        this->imag=obj.imag;           // Copy content
    }

    ~Complex() {
        delete[] ptrReal; // Safe – each object owns its own memory
        ptrReal=NULL;
    }
};
```

Rule of Three: If your class needs a **destructor**, a **copy constructor**, or a **copy assignment operator**, it almost certainly needs all three.

Feature	Shallow Copy	Deep Copy
Memory allocation	Shared (same pointer)	Independent (new memory)
Safe with pointers?	No	Yes
Provided by compiler?	Yes (default)	No (must write manually)
Risk	Dangling pointer / double free	None

4. Operator Overloading

Operator overloading allows you to redefine the behavior of built-in operators (+, -, *, ==, etc.) for user-defined classes.

Why? Built-in types like `int` and `float` already support operators. For custom types, you must define what those operators mean.

```
int num1 = 10, num2 = 20;
int result = num1 + num2; // Works natively

Complex c1(1, 2), c2(3, 4);
// c1 + c2 → doesn't work by default – we must define it!
```

Overloading + Using a Member Function

```
class Complex {
    int real, imag;
public:
    Complex(int r, int i) : real(r), imag(i) {}

    // Overload '+' operator
    Complex operator+(const Complex& obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(1, 2), c2(3, 4);
    Complex c3 = c1 + c2; // Calls c1.operator+(c2)
    c3.display();         // Output: 4 + 6i
}
```

Friend Function for Operator Overloading

Sometimes a member function cannot be used (e.g., when the left operand is not an object of the class). A **friend function** is a non-member function that has access to the private members of a class.

```
class Complex {
    int real, imag;
public:
    Complex(int r, int i) : real(r), imag(i) {}

    // Declare friend function
    friend Complex operator+(const Complex& c1, const Complex& c2);
};
```

```
};

// Define friend function outside the class
Complex operator+(const Complex& c1, const Complex& c2) {
    Complex temp;
    temp.real=c1.real + c2.real,
    temp.imag=c1.imag + c2.imag;
    return temp;
}
```

5. Association

Association describes a relationship between two classes where one class uses or interacts with another. It represents a **"uses-a"** or **"has-a"** relationship.

Types of Association

Type	Description	Example
Association	General use-a relationship	Teacher and Student
Aggregation	Has-a, but independent lifecycle	Department has Employees
Composition	Has-a, dependent lifecycle	House has Rooms

```
class Engine {
public:
    void start() { cout << "Engine started" << endl; }
};

class Car {
    Engine engine; // Composition: Car owns the Engine
public:
    void drive() {
        engine.start();
        cout << "Car is moving" << endl;
    }
};
```

6. Inheritance

Inheritance is an OOP mechanism that allows a new class (derived/child class) to acquire the properties and behaviors of an existing class (base/parent class).

Why? It promotes **code reuse**, **extensibility**, and supports the **"is-a"** relationship.

6.1 Basic Inheritance

```
class Animal {                                // Base class
public:
    string name;
    void eat() { cout << "Eating..." << endl; }
};

class Dog : public Animal {                  // Derived class
public:
    void bark() { cout << "Barking..." << endl; }
};

int main() {
    Dog d;
    d.name = "Tommy";
    d.eat();    // Inherited from Animal
    d.bark();   // Own method
}
```

Inheritance Syntax: `class Derived : access_specifier Base { ... };`

6.2 Protected Data Member

Access Specifier	Inside Class	Derived Class	Outside Class
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

```
class Person {
protected:
    int age;    // Accessible in derived classes, not outside
public:
    string name;
};

class Employee : public Person {
public:
    void setAge(int a) { age = a; } // Can access protected member
    void display() { cout << name << " - " << age; }
};
```

6.3 Modes of Inheritance

The **mode (access specifier)** used during inheritance controls how the base class members are accessible in the derived class.

Base Member	public Inheritance	protected Inheritance	private Inheritance
public	public	protected	private
protected	protected	protected	private
private	Not inherited	Not inherited	Not inherited

```

class Base {
public:    int pub = 1;
protected: int pro = 2;
private: int pri = 3;
};

class PublicDerived : public Base {
    // pub → public, pro → protected, pri → not accessible
};

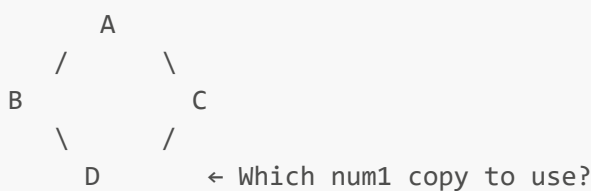
class ProtectedDerived : protected Base {
    // pub → protected, pro → protected, pri → not accessible
};

class PrivateDerived : private Base {
    // pub → private, pro → private, pri → not accessible
};

```

6.4 Diamond Problem

The **diamond problem** occurs in **multiple inheritance** when two parent classes inherit from the same grandparent class, and a child class inherits from both parents. This creates **ambiguity** and **duplicate copies** of the grandparent's data.



```

class A {
public:
    int num1;
};

class B : public A { };
class C : public A { };

```

```
class D : public B, public C {  
    // D has TWO copies of num1!← ERROR: ambiguous!  
};
```

Solution: Virtual Inheritance

```
class B : virtual public A { };  
class C : virtual public A { };  
  
class D : public B, public C {  
    // Now only ONE shared copy of num1  
    // No ambiguity!  
};  
  
;
```
